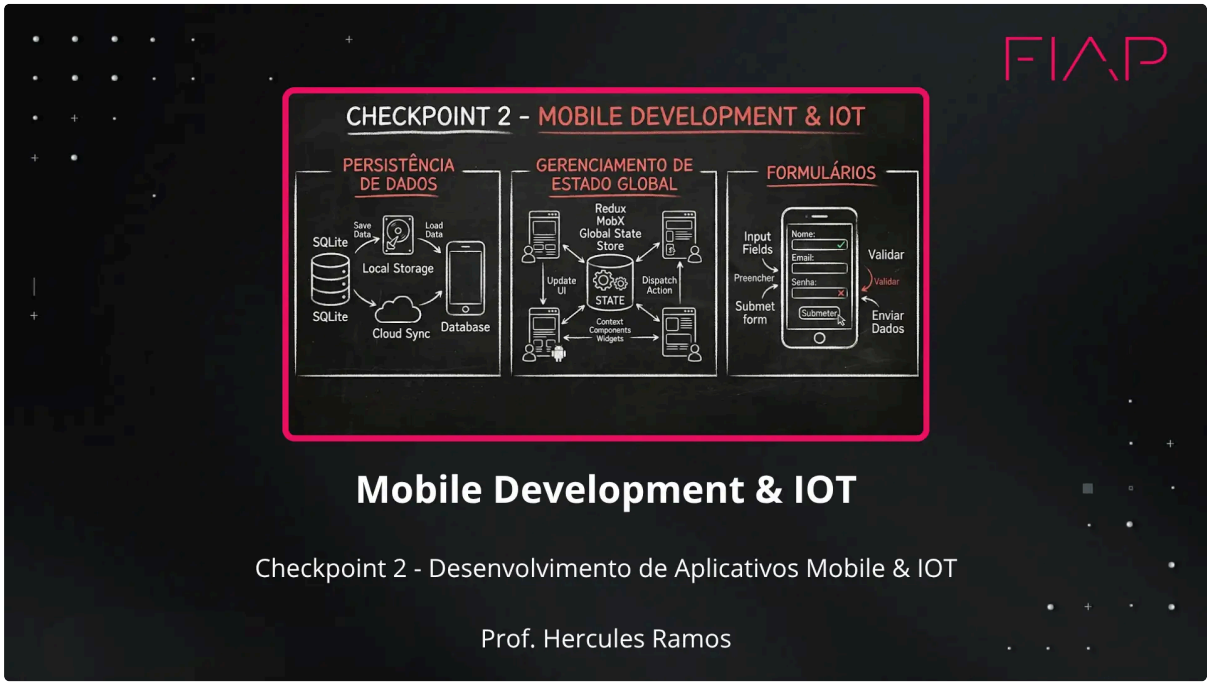
# Checkpoint 2 - Mobile Development & IoT




## Engenharia de Software - 3º Ano

### Mobile Development & IOT

A utilização do uso de soluções para mobile gera o aumento constante da demanda para o desenvolvimento de aplicações multiplaforma. Nesta matéria você será capacitado como um Mobile Developer, atendendo a crescente busca por profissionais com esse perfil. Isso significa ter o controle de múltiplas funções na palma da sua mão entendendo a fundo o conteúdo de desenvolvimento mobile & IoT, e integrando o melhor da sociedade 5.0 com os resultados obtidos da evolução da indústria 4.0.

---

## 📌 Sobre este Checkpoint

O Checkpoint 2 é a **evolução direta do projeto entregue no Checkpoint 1**.

Não estamos começando do zero — estamos elevando o nível. O grupo vai pegar o MVP construído anteriormente e transformá-lo em um app mais completo, com autenticação de usuários, persistência de dados local, gerenciamento de estado global e uma experiência visual refinada.

A lógica é simples: todo produto real passa por iterações. Agora é hora de transformar o protótipo em algo que um usuário real conseguiria (e quereria) usar.

---

## 🎯 Objetivo

Evoluir o aplicativo desenvolvido no CP1, incorporando:

- **Autenticação de usuários** (cadastro e login com dados persistidos)
- **Persistência de dados local** com AsyncStorage
- **Gerenciamento de estado global** via Context API
- **Formulários com validação** de campos obrigatórios e formato

- **Aprimoramento visual e de UX/UI** em todas as telas existentes
- **Pelo menos um diferencial técnico** não abordado em aula, com justificativa documentada

---

## 🛠️ Requisitos Técnicos

O app deve continuar usando **React Native + Expo** e acumular todos os requisitos do CP1. Os novos requisitos são **obrigatórios** e serão avaliados individualmente.

---

### ✅ Obrigatórios — herdados do CP1 (devem permanecer funcionando)

> Qualquer item do CP1 que pare de funcionar no CP2 será descontado na avaliação.

- [ ] Projeto iniciado com Expo CLI e estrutura organizada em pastas (components, screens, context, etc.)
- [ ] Uso de `View`, `Text`, `Image` e `TouchableOpacity`
- [ ] Componentização — sem arquivos monolíticos
- [ ] Estilização com `StyleSheet` — layout consistente e agradável
- [ ] Pelo menos 3 telas distintas com navegação via Expo Router
- [ ] Navegação funcional entre todas as telas (zero telas mortas ou botões sem ação)

---

### 🆕 Obrigatórios — novos para o CP2

#### 1. Autenticação com AsyncStorage

- [ ] **Tela de Cadastro** com os campos:
  - Nome completo
  - E-mail (formato válido: `usuario@dominio.com`)
  - Senha (mínimo 6 caracteres)
  - Confirmação de senha (deve ser idêntica à senha)
- [ ] **Tela de Login** com os campos:
  - E-mail
  - Senha
- [ ] Os dados de usuário cadastrado devem ser **salvos no AsyncStorage**
- [ ] O login deve **validar as credenciais** contra os dados persistidos
- [ ] Após login bem-sucedido, o usuário deve ser redirecionado para a tela principal do app
- [ ] Implementar **logout** com limpeza da sessão e retorno à tela de login
- [ ] O app deve lembrar se o usuário está logado (sessão persistida): ao reabrir o app, o usuário logado não deve precisar fazer login novamente

> ⚠️ **Atenção:** O fluxo de autenticação deve ser completamente funcional. Telas de login/cadastro que não validam nada ou não persistem dados = zero neste critério.

---

#### 2. Persistência de Dados com AsyncStorage

- [ ] Além dos dados de usuário, **os dados funcionais do app** (reservas, itens, agendamentos, etc.) também devem ser persistidos com AsyncStorage
- [ ] Os dados devem sobreviver ao fechamento e reabertura do app

- [ ] Implementar leitura dos dados ao montar as telas (com `useEffect`)
- [ ] Implementar atualização dos dados sempre que houver inserção, edição ou exclusão

---

### 3. Gerenciamento de Estado Global com Context API

- [ ] Criar **pelo menos um Context** para gerenciar o estado compartilhado entre telas (ex: `AuthContext`, `AppDataContext`)
- [ ] O `AuthContext` deve disponibilizar: usuário logado, função de login, função de logout
- [ ] O acesso a telas autenticadas deve ser protegido — usuário não logado não pode acessar o conteúdo principal do app
- [ ] O estado global deve ser consumido via `useContext` nos componentes que precisarem

---

### 4. Formulários com Validação

- [ ] **Todos os formulários** do app (cadastro, login e qualquer outro formulário existente) devem ter validação explícita dos campos
- [ ] Validações obrigatórias mínimas:
  - Campo vazio → mensagem de erro inline (ex: `"O e-mail é obrigatório"`)
  - Formato de e-mail inválido → mensagem de erro inline
  - Senha com menos de 6 caracteres → mensagem de erro inline
  - Senhas divergentes no cadastro → mensagem de erro inline
- [ ] Os erros devem aparecer **abaixo do campo correspondente**, em vermelho, sem alert/modal
- [ ] O botão de submissão não deve funcionar enquanto há erros de validação

---

### ⭐ Diferenciais — pelo menos UM é obrigatório

O grupo deve implementar **no mínimo um** diferencial. O diferencial escolhido deve ser **justificado no README** com a explicação de por que foi escolhido e o que agrega ao projeto.

> ⚠️ **Obrigatório:** grupos que não implementarem nenhum diferencial terão desconto direto na nota, independentemente da qualidade do restante.

| # | Diferencial | Descrição |
|---|---|---|
| 1 | **Expo SecureStore** | Armazenar o token/sessão de forma segura em vez de AsyncStorage puro |
| 2 | **Animações com Animated API** | Transições e feedbacks visuais animados (ex: shake no erro de login, fade-in de telas) |
| 3 | **Modo Escuro / Tema dinâmico** | Suporte a dark mode com alternância pelo usuário, usando Context |
| 4 | **Notificações locais com Expo Notifications** | Notificações agendadas (ex: lembrete de reserva, prazo de entrega) |
| 5 | **Câmera ou galeria com Expo ImagePicker** | Upload de foto de perfil ou imagem relacionada ao app |
| 6 | **Busca e filtragem em tempo real** | FlatList com campo de busca e filtro dinâmico por texto |

| # | Diferencial | Descrição |
|---|---|---|
| 7 | **Outro** | Proposta justificada pelo grupo |

---

## 🎨 Requisitos de UX/UI

A qualidade visual **é critério de avaliação** e será avaliada com mais rigor do que no CP1.

### Obrigatório em todas as telas:

- [ ] **Hierarquia visual clara**: títulos, subtítulos e corpo de texto com tamanhos e pesos distintos
- [ ] **Paleta de cores coerente**: no mínimo uma cor primária, uma cor de destaque e cores de feedback (verde para sucesso, vermelho para erro)
- [ ] **Espaçamento consistente**: padding e margin uniformes — nada "grudado" nas bordas
- [ ] **Componentes de feedback visual**:
  - Loading spinner (ActivityIndicator) durante operações assíncronas
  - Mensagem de erro inline nos formulários (não usar `Alert`)
  - Mensagem de sucesso após ação concluída
  - Tela ou componente de "lista vazia" quando não há dados
- [ ] **Teclado não deve cobrir campos de formulário**: usar `KeyboardAvoidingView` ou `ScrollView`

### Fortemente recomendado (valorizado na avaliação):

- [ ] Identidade visual coerente com a FIAP ou com o tema do app
- [ ] Ícones consistentes (ex: `@expo/vector-icons`)
- [ ] Inputs com placeholder, label visível e ícone indicativo
- [ ] Responsividade mínima com `Dimensions` ou Flexbox bem estruturado

---

## 📁 Estrutura de Pastas Sugerida

O projeto deve ter uma organização clara. A estrutura abaixo é uma referência — o grupo pode adaptar, desde que a organização esteja documentada no README.

```
meu-app/
├── app/                    # Rotas do Expo Router
│   ├── (auth)/
│   │   ├── login.jsx
│   │   └── cadastro.jsx
│   ├── (tabs)/
│   │   ├── index.jsx
│   │   └── ...
│   └── _layout.jsx
├── components/             # Componentes reutilizáveis
│   ├── Button.jsx
│   ├── Input.jsx
│   └── ...
├── context/                # Contexts globais
│   ├── AuthContext.jsx
│   └── AppDataContext.jsx
├── hooks/                  # Custom hooks (se houver)
```

```
├── constants/              # Cores, tamanhos, textos fixos
└── assets/                 # Imagens e fontes
```

---

## 📋 Requisitos de Entrega

### 1. Repositório GitHub

- [ ] Repositório **público**
- [ ] Nome do repositório: `fiap-mdi-cp2-[nome-do-app]` (ex: `fiap-mdi-cp2-cantina-app`)
- [ ] **Todos os membros** devem ter commits relevantes — histórico será verificado commit a commit
- [ ] Mensagens de commit descritivas em português ou inglês (commits como `"fix"`, `"update"`, `"asdfsdf"` serão penalizados)
- [ ] O projeto deve rodar sem erros com `npx expo start`
- [ ] Branch `main` deve sempre conter a versão funcional mais recente

---

### 2. README.md — Documentação Completa (peso alto!)

O README é a **vitrine do projeto**. Um README incompleto impacta diretamente a nota de Documentação.

#### a) Sobre o Projeto

- Nome do app e descrição do problema que resolve
- Qual operação da FIAP foi escolhida e por quê
- O que mudou/melhorou em relação ao CP1
- Lista completa de funcionalidades implementadas

#### b) Integrantes do Grupo

- Nome completo de cada integrante
- RM de cada integrante

#### c) Como Rodar o Projeto

- Pré-requisitos (Node, Expo Go, versão do Expo SDK)
- Passo a passo para clonar e executar localmente:

```
git clone https://github.com/usuario/fiap-mdi-cp2-nome-do-app
cd fiap-mdi-cp2-nome-do-app
npm install
npx expo start
```

#### d) Demonstração Visual — OBRIGATÓRIO

- **Prints de TODAS as telas** do app (mínimo: uma print por tela, incluindo telas de login e cadastro)
- **GIF ou vídeo** demonstrando o fluxo completo: cadastro → login → uso do app → logout

> 💡 Para gravar GIF: Android Studio → emulador → *Record Screen*.
>
> Para vídeo: grave a tela do celular ou emulador e suba no YouTube/Google Drive com link no README.
>
> ⛔ README sem prints e sem demonstração em vídeo/GIF = desconto automático de 50% na nota de Documentação.

#### e) Decisões Técnicas

- Como o projeto foi estruturado (pastas e responsabilidades)
- Quais Contexts foram criados e o que cada um gerencia
- Como a autenticação foi implementada
- Como o AsyncStorage foi utilizado (quais dados são persistidos e com quais chaves)
- Como a navegação protegida foi implementada

### f) Diferencial Implementado — OBRIGATÓRIO

- Qual diferencial foi escolhido
- **Justificativa**: por que o grupo escolheu esse diferencial e o que ele agrega à experiência do usuário
- Como foi implementado (resumo técnico)

### g) Próximos Passos *(opcional, mas valorizado)*

- O que o grupo implementaria com mais tempo

---

## 👥 Composição dos Grupos

- Grupos de **4 a 5 integrantes** (mesmos grupos do CP1)
- Não serão aceitos grupos fora do tamanho especificado sem autorização prévia
- Grupos com integrantes sem commits relevantes terão desconto proporcional na nota de Colaboração

---

## 📊 Critérios de Avaliação

| Critério | Descrição | Peso |
|---|---|---|
| **Funcionalidade** | O app roda sem erros? Auth, persistência, validação e navegação funcionam corretamente? | 25% |
| **Qualidade Técnica** | Uso adequado de Context API, AsyncStorage, hooks, componentes e navegação protegida | 20% |
| **Apresentação** | Clareza na exposição do problema, demonstração ao vivo do app e domínio técnico nas respostas | 15% |
| **Documentação** | README completo, com prints de todas as telas, vídeo/GIF e justificativa do diferencial | 15% |
| **UX & Design** | Interface intuitiva, consistente, com feedback visual adequado e hierarquia visual clara | 15% |
| **Colaboração no Git** | Todos os membros têm commits relevantes e descritivos | 5% |
| **Diferencial** | Implementação além do mínimo, justificada e funcional | 5% |

---

## ⛔ O que NÃO será aceito

- Apps que não inicializam ou travam na abertura
- Repositório privado no momento da avaliação
- README vazio, genérico ou sem demonstração visual (prints + vídeo/GIF)
- Autenticação "fake" — login que aceita qualquer entrada sem validar

- AsyncStorage usado apenas para um único dado (ex: só o token, mas nada funcional do app)
- Context API importado mas não utilizado de forma significativa
- Formulários sem validação ou com validação apenas via `Alert`
- Projetos individuais entregues como grupo (verificado via commits)
- Cópias de tutoriais sem adaptação ao tema proposto
- Grupo sem nenhum diferencial implementado
- Uso de bibliotecas externas de autenticação (Firebase Auth, Supabase, etc.) — o foco é AsyncStorage e Context API nativos

> 🔴 **Atenção total:** Projeto que não roda = **zero em Funcionalidade**. README sem prints = **-50% em Documentação**. Membro sem commits = **desconto individual em Colaboração**.

---

## 📅 Prazo e Submissão

| Etapa | Data |
|---|---|
| Alinhamento do escopo com o professor | Definido em aula |
| Entrega do link do repositório (Teams) | Definido em aula |
| Apresentação (se aplicável) | Definido em aula |

A submissão deve ser feita como **Trabalho pelo Teams** com o link do repositório GitHub.

---

## ❓ Dúvidas?

Tragam para a aula! Quanto antes o grupo alinhar o escopo e o diferencial escolhido com o professor, menores as chances de surpresas na entrega.

> *"Um bom produto não é o que tem mais features — é o que resolve o problema certo, da forma certa, para o usuário certo."*

---

## Bom trabalho, galera. 🚀

**Contato:**

📧 profhercules.ramos@fiap.com.br

💼 LinkedIn

**#KeepCoding #ReactNative #FIAP**

---

**Let's code!** 💻🔥

*"O único modo de aprender programação é programando."* - Todo desenvolvedor, sempre

---

Copyright © 2026 Prof. Hercules Ramos

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).